

BOX DIAMONDS — Architecture & Phased Build Plan

Context

This is a greenfield project (empty folder at `/Users/jamsheerpk/box-gold`). The user has provided a very detailed 46-point product/engineering spec for a premium jewellery ecommerce platform ("BOX DIAMONDS") and explicitly asked for **architecture + plan first, no code yet**, with backend, admin, and customer web kept as separate apps, and every subsequent feature area to be discussed and built **one section at a time**, not all at once.

Decisions confirmed with the user before finalizing this plan:

- **Repo structure:** single monorepo, 3 app folders (`backend`, `web`, `admin`) + a shared package.
- **Payment gateway:** not chosen yet — build a gateway-agnostic `PaymentService` abstraction with a mock/stub provider; swap in Razorpay/Cashfree/Stripe later without touching checkout logic.
- **OTP SMS provider:** MSG91, behind an `OTPProvider` abstraction (swappable).
- **AI image generation provider:** not chosen yet — build the async job architecture and admin approval workflow now with a stub/mock image generator; swap in a real model provider later.
- **Reviews:** full customer-submitted reviews, gated to verified purchases (order must be `DELIVERED`), with an admin moderation queue.
- **Live gold pricing:** gold price is auto-pricing — a scheduled sync job fetches a live rate and automatically recalculates affected product prices (admin can still lock a specific product's price). Rate source not chosen yet — build a swappable `MetalRateProvider` abstraction with a stub. Diamond pricing is a single global admin-managed ₹-per-carat rate (with per-product manual override). Purities supported: 14K/18K/22K/24K, gold only for live pricing — platinum is priced manually like diamonds.

Corrections mandated on 2026-08-10 (applied throughout this revision):

- **SEO rendering:** public pages (Home, PLP/category, Collections, PDP) get request-time SSR with a Postgres-backed page cache + invalidation, not build-time-only static generation — chosen specifically because build-time generation would go stale every time the live gold rate updates prices. See §1a.
- **Order/payment split:** checkout creates a `PENDING_PAYMENT` order + inventory reservation; only a signature-verified, idempotent payment webhook confirms the order and finalizes stock/coupon usage. See revised §11.
- **Inventory reservation:** a `stock_reservations` table with expiry prevents overselling low-stock/one-of-a-kind pieces between "added to order" and "payment confirmed." See revised §11.
- **ShippingProvider abstraction:** stub now, real courier integration later — mirrors the OTP/Payment/AI-Image provider pattern. See §11b.
- **Hierarchical categories:** `categories.parent_id` (self-referencing FK) replaces the flat category assumption.
- **Default address:** enforced via a real Postgres partial unique index, not application logic.

- **Verified-review gating:** uses `orders.status = 'DELIVERED'` (the order-level fulfillment status that already exists) rather than an invented per-item field — flagged for the user to override if per-item fulfillment is actually needed later.
- **AIImageProvider interface:** restructured from `Buffer[]` to structured results supporting URLs, buffers, provider job IDs, metadata, and async/polling providers. See revised §10.
- **Performance/SEO/security are per-phase acceptance criteria**, not solely a final-phase audit — see the tags on the Phased Build Plan.

This plan defines the target architecture once, so every future phase has a shared reference to build against — it is not itself an instruction to build everything now.

1. Monorepo Structure

```

box-gold/
├── backend/                                # Node.js + Express + PostgreSQL REST API
│   ├── src/
│   │   ├── config/                        # env loading, db pool, constants
│   │   ├── routes/                       # thin route definitions per module
│   │   ├── controllers/                  # request/response handling only
│   │   ├── services/                     # business logic (OTPService, PaymentService, CartService...)
│   │   ├── repositories/                 # SQL queries / data access, one per table group
│   │   ├── validators/                   # zod schemas per endpoint
│   │   ├── middleware/                   # auth, admin-auth, rate-limit, error handler, csrf
│   │   ├── jobs/                         # AI image worker, email worker (pg-boss handlers)
│   │   ├── providers/                    # OTPProvider (MSG91 + stub), PaymentProvider (stub), AIImageP
│   │   ├── db/
│   │   │   ├── migrations/
│   │   │   └── seeds/
│   │   └── utils/
│   ├── tests/
│   └── package.json
├── web/                                    # Customer storefront – React + Vite + React Router
│   ├── src/
│   │   ├── pages/                        # route-level components (lazy-loaded)
│   │   ├── components/                   # reusable UI (Card, Badge, PriceBreakup, etc.)
│   │   ├── layouts/
│   │   ├── features/                     # cart, wishlist, auth, search – colocated logic+UI
│   │   ├── store/                        # Zustand slices (cart, wishlist, auth session)
│   │   ├── api/                          # typed fetch client, one module per backend REST group
│   │   ├── styles/                       # design tokens (colors, type scale), CSS Modules
│   │   ├── entry-client.tsx              # hydrateRoot – every route
│   │   └── entry-server.tsx              # renderToString – SSR routes only (Home/PLP/Collections/PDP)
│   ├── server/                           # SSR render server: request-time render, page_cache read/wri
│   │   └── index.js
│   └── package.json
├── admin/                                 # Admin panel – React + Vite + React Router (separate app)
│   ├── src/ (same shape as web: pages/components/features/store/api)
│   └── package.json
├── packages/
│   └── shared/                            # shared TS types, zod schemas, constants (order status enums,
└── README.md

```

Two independent Vite apps (`web`, `admin`) keep the customer bundle free of admin-only code, satisfying the "minimal JS bundle" performance requirement. `packages/shared` avoids type drift between backend and both frontends without coupling deploys.

1a. Rendering Architecture (SSR + SEO)

Confirmed with the user: request-time SSR with cache + invalidation, not build-time-only static generation — because gold-price auto-updates mean a page rendered once at `vite build` time would go stale until the next deploy.

Scope: SSR applies only to the four public SEO surfaces — Home, PLP/category, Collections, PDP. Cart, checkout, account, search-results, and the entire `admin` app stay pure client-side rendering (not indexed, behind auth or low SEO value, no reason to pay the SSR cost).

Runtime shape: `web` ships two things in production — the built static client bundle (JS/CSS, served from a CDN/static host) and a small persistent Node process (`web/server`) that renders those four route types to HTML per request using Vite's SSR build (`entry-server.tsx`, `renderToString` / `renderToPipeableStream`) and serves it directly (gzip/brotli compressed). This is the one deliberate change from "static SPA" — those four route types can no longer be served from a plain static host alone.

Data fetching during SSR: the SSR server calls the same public `backend` REST endpoints the client would (`/api/v1/products`, `/api/v1/homepage`, etc. — no duplicated business logic), fetches into a TanStack Query `QueryClient`, then dehydrates that cache into the HTML (`window.__REACT_QUERY_STATE__`). `entry-client.tsx` rehydrates it on `hydrateRoot`, so there's no double-fetch flash on first load. If a backend call fails/times out during render, the server falls back to a minimal shell (200 response, no SSR content) rather than a 500 — the client then fetches normally.

Page cache — `page_cache` table (Postgres, no Redis, consistent with the rest of the stack):

```
page_cache (url PK, html, generated_at)
```

- On request: SSR server checks `page_cache` for a row newer than `SSR_CACHE_TTL_SECONDS` (default 10 min) — cache hit serves the stored HTML directly (fast, no render work); miss/stale triggers a fresh render, which is stored back before responding.
- The TTL alone bounds worst-case staleness even with zero explicit invalidation (gold syncs happen a few times a day, not every minute).
- Explicit invalidation** for instant feedback: any `backend` mutation that affects a public page — product publish/update, price recalculation (§9a), category/collection change, homepage section change — deletes the relevant `page_cache` rows (the product's own PDP URL + its PLP/category URL + Home if featured) in the same service call. Since both `backend` and `web/server` read/write the same Postgres database, this is a plain indexed `DELETE`, no cross-process messaging needed. This is a deliberately scoped exception to "web only talks to backend via REST" — `web/server` gets its own narrow Postgres connection limited to this one cache table, nothing else.

What has to be in the initial HTML: `<title>`, meta description, canonical URL, OpenGraph tags, and JSON-LD (`Product` on PDP, `BreadcrumbList` everywhere, `Organization` sitewide) are rendered server-side into `<head>` — this is the actual SEO-critical content the correction is about, not just visible page text.

2. Tech Choices (kept deliberately minimal)

Concern	Choice	Why
Frontend state (client)	Zustand	~1KB, no boilerplate, used for cart/wishlist/session only
Frontend state (server/cache)	TanStack Query	Handles pagination, caching, loading/error states — reduces custom code and rerenders
Styling	CSS Modules + a small design-tokens file	No heavy UI kit; matches "avoid heavy UI libraries"
Validation	zod	Shared between backend validators and <code>packages/shared</code>
Sessions	<code>express-session</code> + <code>connect-pg-simple</code>	HttpOnly cookies backed by Postgres — no Redis needed, stays inside the declared stack
Background jobs (AI images, emails)	<code>pg-boss</code>	Postgres-backed job queue — reliable retries/scheduling without adding Redis/infra
Image processing	<code>sharp</code>	Thumbnail/AVIF/WebP generation, industry standard, fast
Object storage	<code>StorageProvider</code> abstraction — local disk in dev, S3-compatible (S3/R2/Spaces) in prod	CDN-ready, swappable
Email	<code>EmailProvider</code> abstraction behind <code>email_jobs</code> queue (e.g. SES/Postmark later)	Async, never blocks checkout

No Redis, no ORM (raw SQL via `pg` + repositories, for full control over indexes/query shape — matches "proper database indexing" and "optimized PostgreSQL queries" priorities). Still no Next.js — SSR for the public storefront routes is hand-rolled on top of Vite's own SSR build support (see §1a) rather than adopting a framework, keeping the stack as close to plain React + Vite + React Router as the SEO requirement allows.

3. Database Schema (PostgreSQL)

Tables as listed by the user in section 38, organized into logical groups. Key structural decisions:

- **Identity/auth:** `users` (customers, OTP-based), `admin_users` + `admin_roles` (separate, password-based) are fully separate tables/flows — never share auth logic.

- **OTP:** `otp_verifications` is standalone (`mobile_number`, `otp_hash`, `purpose` enum, `attempts`, `expires_at`, `verified_at`) — never touches `users.password`. Index on (`mobile_number`, `purpose`, `created_at`) for throttling lookups.
- **Sessions:** `customer_sessions` (or rely on `connect-pg-simple`'s own session table) keyed by session id, with `user_id`, `expires_at`.
- **Products:** `products` holds all jewellery attributes from section 7 + `status` enum (`DRAFT`, `AI_PROCESSING`, `AI_READY`, `PUBLISHED`, `FAILED`) + pricing fields: `metal_type` (`GOLD` / `PLATINUM` / ...), `purity` (`14K` / `18K` / `22K` / `24K`, gold only), `gross_weight_grams`, `net_weight_grams`, `diamond_weight_carats`, `gold_value` (auto-computed, cached), `diamond_value` (computed from global rate unless `diamond_value_is_manual`), `making_charge` (admin), `gst_percent` (admin), `mrp`, `selling_price` (computed total, admin-overridable), `is_price_locked` (boolean — excludes this product from automatic rate-sync recalculation). `product_images` has `type` enum (`ORIGINAL`, `AI_GENERATED`), `variant` (thumbnail/small/medium/large/original paths), `sort_order`, `is_primary`.
- **Pricing engine:** `gold_rates` (`purity`, `rate_per_gram`, `source`, `fetches_at`) — one row per purity per sync, latest row per purity is "current"; `diamond_rates` (`rate_per_carat`, `set_by_admin_id`, `created_at`) — admin-managed global per-carat rate, latest row is "current"; `product_price_history` (`product_id`, `old_selling_price`, `new_selling_price`, `gold_rate_id`, `reason` [`RATE_SYNC` / `ADMIN_MANUAL` / `DIAMOND_RATE_CHANGE`], `created_at`) — audit trail for every automatic or manual price change, indexed on `product_id`. See §9a.
- **Catalog:** `categories` (now hierarchical — self-referencing `parent_id`, nullable, indexed — enables subcategories like Rings → Diamond Rings → Solitaire Rings; breadcrumbs and nested filter trees are built with a `WITH RECURSIVE` CTE walking `parent_id`, no need for a heavier tree extension at this depth), `collections`, `product_categories` (many-to-many). Indexes on `slug` (unique) for SEO URLs, `parent_id`, and on `category_id`, `status`, `created_at` for listing queries.
- **Commerce:** `carts` / `cart_items` support both guest (session-id keyed) and logged-in (user-id keyed) carts, merged on login. `wishlist` / `wishlist_items` mirror the same pattern.
- **Inventory:** `stock_reservations` (`product_id`, `order_id`, `quantity`, `status` [`ACTIVE` / `CONFIRMED` / `RELEASED`], `expires_at`, `created_at`) — created at checkout (default TTL 15 min, configurable via `RESERVATION_TTL_MINUTES`), released by a scheduled sweep job if payment never confirms, converted to a permanent stock decrement by the payment webhook on success. Available stock for display/purchase-time checks = `products.stock_quantity - SUM(active reservations for that product)`. Indexed on (`product_id`, `status`) for the availability calc and (`expires_at`, `status`) for the sweep. See revised §11.
- **Orders:** `orders`, `order_items`, `order_status_history` (append-only audit of the state machine in §11 below), `payments` (gateway-agnostic: `provider`, `provider_ref` UNIQUE — the gateway's own transaction id, giving webhook idempotency for free via a conflict check — `status`, `raw_payload` jsonb), `shipments` (`provider`, `provider_shipment_id`, `tracking_number`, `courier_name`, `status`, `raw_webhook_payload` jsonb — see §11b).
- **Addresses:** `addresses` (`user_id`, `type` [Home/Office/Other], `is_default`, `name/mobile/line/building/landmark/city/state/pincode/country`) — indexed on `user_id`. One default per user is enforced at the database level: `CREATE UNIQUE INDEX addresses_one_default_per_user ON`

`addresses(user_id) WHERE is_default = true;` — a real Postgres partial unique index, not application logic, so it can't race.

- **Rendering:** `page_cache` (`url` PK, `html`, `generated_at`) — SSR render cache for the four public SEO route types, read/written by both `backend` (invalidation on mutation) and `web/server` (render + serve). See §1a.
- **Coupons:** `coupons` (`code` unique, discount type/value, validity window, min order value, usage limits), `coupon_usage` (`coupon_id`, `user_id`, `order_id`) — indexed on `code` and (`coupon_id`, `user_id`) to enforce per-user usage limits.
- **Homepage:** `homepage_sections` (`type`, `heading`, `is_enabled`, `sort_order`), `homepage_items` (`section_id`, `image`, `cta`, links to `category_id` / `collection_id` / `product_id`, `sort_order`) — public homepage GET reads only enabled sections/items ordered by `sort_order`.
- **Reviews:** `reviews` (`product_id`, `user_id`, `order_item_id` nullable, `rating` 1-5, `title`, `body`, `is_verified_purchase`, `status` [`PENDING`, `APPROVED`, `REJECTED`], `created_at`) — indexed on (`product_id`, `status`) for PDP display and (`user_id`) for "my reviews". `order_item_id` links a review to the specific order item purchased; "verified purchase" is enforced server-side against the real fulfillment field that already exists — `orders.status = 'DELIVERED'` on the parent order (not an invented per-item status field, since this store has one shipment/one delivery method per order — flag if per-item fulfillment granularity is actually needed later). Product's average rating is denormalized onto `products.rating_avg` / `rating_count`, updated transactionally on approve — avoids a join/aggregate per listing row on the PLP.
- **Jobs:** `ai_image_jobs` (`product_id`, `status`, `provider`, `error`), `ai_generated_images` (`job_id`, `image_url`, `image_buffer` nullable, `provider_job_id`, `metadata jsonb`, `approved boolean`), `email_jobs` / `email_logs` — all consumed by `pg-boss`.
- **Admin:** `audit_logs` (`admin_user_id`, `action`, `entity`, `entity_id`, `diff jsonb`, `created_at`) for every admin mutation.
- **Indexing strategy:** every foreign key gets an index; listing/filter queries (`products` by category+status+price, `orders` by user+status) get composite indexes matching actual filter/sort combinations; all lookup-by-slug/mobile/sku columns are unique-indexed.

Full column-level DDL will be written as actual migrations in Phase 1 of the build (not in this planning doc) so it can be reviewed against real migration tooling (`node-pg-migrate` recommended — lightweight, SQL-based, no ORM lock-in).

4. Customer Route List (web)

/	Homepage
/rings /earrings /necklaces /pendants /bangles /bracelets /gifts	PLP (category slug)
/collections/:slug	PLP (collection)
/:categorySlug/:productSlug	PDP e.g. /rings/celeste-diamond-ring
/search?q=...	Search results
/cart	
/wishlist	
/checkout	(contact → address → delivery → payment → summary, single-page)
/order-confirmation/:orderId	
/account	(redirects to /account/orders)
/account/orders	
/account/orders/:orderId	Track order
/account/addresses	
/account/profile	
/login	Mobile OTP (also invoked as a modal/drawer from header + chec

5. Admin Route List (admin)

/login	Admin email+password
/dashboard	
/products /products/new /products/:id/edit	(incl. AI image workflow tab)
/categories	
/collections	
/orders /orders/:id	
/customers /customers/:id	
/coupons	
/homepage	Section manager (drag-reorder, enable/disable, content)
/payments	Gateway config/status (once a real provider is chosen)
/shipping	Per-order shipment status, manual "Ship"/"Cancel Shipment" ac
/email-templates	
/ai-image-jobs	Queue monitor: pending/processing/ready/failed, approve/regen
/reviews	Moderation queue: pending/approved/rejected
/pricing	Gold rate history, manual sync, diamond rate setting
/settings	
/admin-users	
/audit-logs	

6. REST API Plan

```

/api/v1/auth/admin/login, /logout, /me
/api/v1/otp/send, /verify           purpose: LOGIN | SIGNUP | CHECKOUT | PHONE_CH
/api/v1/customers/me, /me/addresses...
/api/v1/products                   GET (paginated, filterable), GET /:slug
/api/v1/categories, /api/v1/collections
/api/v1/search                     ?q= (debounced client-side, indexed ILIKE/trigram or tsvector)
/api/v1/cart                       GET/POST/PATCH/DELETE items, works for guest via cart-session c
/api/v1/wishlist                   same guest/user pattern
/api/v1/addresses
/api/v1/checkout                   creates order draft, attaches verified contact
/api/v1/orders, /api/v1/orders/:id, /:id/track
/api/v1/payments/webhook           signature-verified, idempotent (dedupe on payments.provider_re
/api/v1/shipping/webhook           signature-verified, idempotent (courier tracking/status update
/api/v1/coupons/apply
/api/v1/homepage                   public GET of active, ordered sections
/api/v1/products/:id/reviews       GET (approved only, paginated), POST (auth + verified-purchase

/api/v1/admin/products (+ /:id/ai-images, /:id/ai-images/:imageId/approve|reject|regenerate)
/api/v1/admin/categories, /collections, /orders, /customers, /coupons
/api/v1/admin/homepage             CRUD + reorder
/api/v1/admin/email-templates
/api/v1/admin/ai-image-jobs
/api/v1/admin/reviews              moderation queue: approve/reject
/api/v1/admin/pricing/gold-rates   GET history, POST /sync (manual trigger)
/api/v1/admin/pricing/diamond-rate GET current, PUT update
/api/v1/admin/pricing/preview      POST – live gold/diamond value calc for the product f
/api/v1/admin/products/:id/price-lock PATCH – toggle exclusion from auto rate-sync
/api/v1/admin/shipping/orders/:id/ship POST – creates shipment via ShippingProvider
/api/v1/admin/shipping/orders/:id/cancel-shipment POST
/api/v1/admin/settings, /admin-users, /audit-logs

```

web/server's `page_cache` reads/writes are internal to that process (direct Postgres access, not a public or admin REST surface) — see §1a.

All list endpoints are cursor/offset paginated (`?page=&limit=`) — no unbounded queries, matching the performance mandate.

7. Mobile OTP Auth Flow

```
POST /otp/send {mobile, purpose}
  → normalize to +91XXXXXXXXXX → throttle check (per-mobile + per-IP, both windowed
    via otp_verifications COUNT queries) → generate 6-digit OTP → bcrypt-hash it →
    insert otp_verifications row (expires_at = now+5min) → OTPProvider.sendOTP()
    (MSG91 in prod, console-log stub in dev) → 202 response, OTP never in payload/logs

POST /otp/verify {mobile, otp, purpose}
  → fetch latest non-expired, non-verified row for mobile+purpose → attempts < max?
  → compare hash → on success: verified_at=now, mark row consumed →
    LOGIN/SIGNUP: find-or-create user by mobile → create session (HttpOnly cookie)
    CHECKOUT: mark checkout session as phone-verified, no new login session forced
  → on failure: attempts += 1, generic error, lock after N failures (configurable)
```

`OTPProvider` interface (`sendOTP(mobile, code)`), name only — no `verifyOTP` on the provider since verification is local against the hash) lets MSG91 be swapped for Twilio/SNS by changing one binding in `providers/otp/`. All limits (expiry, cooldown, max attempts, hourly/daily caps) come from env vars, not hardcoded.

8. Session & Security Architecture

- Customer session: `HttpOnly`, `Secure` (prod), `SameSite=Lax` cookie holding a signed session id; row in Postgres session store; sliding expiry.
- Admin session: same mechanism, separate cookie name/namespace, role claim checked by `requireRole()` middleware on every `/admin` route.
- CSRF: `SameSite=Lax` + custom header (`X-Requested-With`) check on state-changing requests is sufficient for a same-site SPA+API; add a double-submit token only if a cross-site payment redirect flow needs it.
- Rate limiting: `express-rate-limit` (IP-based, general API abuse) + custom Postgres-backed throttling specifically for OTP endpoints (needs to survive restarts and work per-mobile, not just per-IP).
- Input validation at every route boundary via `zod`; parameterized SQL everywhere (no string-concatenated queries); `helmet` for security headers; strict file MIME/size checks on image upload; webhook signature verification before ever marking a payment `paid`.

9. Product & Image Architecture

Upload flow: original saved as-is (authoritative, never modified) → `sharp` derives `thumbnail/small/medium/large` variants in AVIF+WebP (with a JPEG fallback) → all variants + original pushed through `StorageProvider` → `product_images` rows created with `variant`, `format`, `type=ORIGINAL`. Listing pages request `small`, PDP requests `large`, `srcset / sizes` handles responsive

selection, nothing above `small` loads on scroll-not-yet-visible cards (native `loading="lazy"` + Intersection Observer fallback where finer control is needed).

9a. Live Pricing Engine (Gold Auto-Pricing + Diamond Manual Pricing)

Confirmed with the user: gold price is auto-pricing (recalculates automatically as the live rate changes), diamond price is a single global admin-managed rate per carat (with per-product manual override still possible), purity support is 14K/18K/22K/24K, and only gold is live-priced — platinum products get an admin-set value like diamonds do, since the spec doesn't require a live platinum feed. The gold-rate source itself is not chosen yet — build a swappable `MetalRateProvider` abstraction with a stub now, exactly like the OTP/payment/AI providers.

PricingService (backend/src/services/pricingService.js):

- `computeGoldValue(netWeightGrams, purity) → netWeightGrams × currentGoldRate(purity)`, where `currentGoldRate` reads the latest `gold_rates` row for that purity.
- `computeDiamondValue(diamondWeightCarats, manualOverride?) → manualOverride ?? diamondWeightCarats × currentDiamondRate()`.
- `computeSellingPrice(product) → goldValue + diamondValue + makingCharge + gst; mrp` stays independently admin-settable (equal to selling price by default, raised to show a discount%).

Rate sync flow (scheduled, pg-boss cron job — interval configurable via `GOLD_RATE_SYNC_CRON`):

```
MetalRateProvider.fetchRate('GOLD') [stub now, returns a mock 24K rate/gram]
→ derive 14K/18K/22K rates from the 24K rate via a purity multiplier
  (default karat/24, overridable in settings if market convention differs)
→ insert 4 new gold_rates rows
→ enqueue recalculate-gold-prices job
  → for every product where metal_type=GOLD AND is_price_locked=false:
    recompute gold_value from net_weight_grams × new rate,
    recompute selling_price, write a product_price_history row,
    update the product row — batched in transactions
```

A manual "Sync Now" trigger (`POST /api/v1/admin/pricing/gold-rates/sync`) runs the same flow on demand. Diamond rate changes (`PUT /api/v1/admin/pricing/diamond-rate`) trigger an equivalent recalculation pass scoped to products with `diamond_weight_carats > 0` and no manual diamond override.

Admin Add/Edit Product — live preview: as soon as the admin picks `metal_type` + `purity` + enters `net_weight_grams` (and `diamond_weight_carats` if applicable), the form calls `POST /api/v1/admin/pricing/preview` with those fields and gets back the live-calculated `gold_value` / `diamond_value` / suggested `selling_price` using current rates — admin can accept it or override `making_charge` / `gst_percent` / `mrp` / `selling_price` before saving. This is what satisfies "choose purity and calculate with live pricing" when adding a product.

Order-time price snapshot: cart and PDP always show the live current computed price (it can change day to day). The moment checkout creates an order, `order_items` snapshot the exact `gold_value`, `diamond_value`, `making_charge`, `gst`, `selling_price`, and the `gold_rate_id` that produced them — later rate syncs never retroactively alter a placed order, only live listings/PDP pricing going forward.

New admin surface: `/pricing` route (gold rate history + manual sync + diamond rate setting), plus a per-product "lock price" toggle (`PATCH /api/v1/admin/products/:id/price-lock`) so a specific piece can be excluded from automatic recalculation if the admin wants to hold its price steady.

10. AI Image Job Architecture

```
Admin uploads original → product saved as DRAFT → ai_image_jobs row created (status=PENDING)
→ 201 response returned immediately (no blocking) → pg-boss worker picks job up
→ product.status = AI_PROCESSING → AIImageProvider.startGeneration(originalImage)
→ worker polls getGenerationStatus(providerJobId) until each result reports READY/FAILED
  (or the provider posts to a webhook instead of being polled — same end state either way)
→ ai_generated_images rows inserted from the structured results → product.status = AI_READ
→ admin notified (poll or simple in-app badge)
→ admin approves/rejects/regenerates individual images → chooses primary → reorders
  gallery → product.status = PUBLISHED
```

AIImageProvider interface (corrected from a flat `generate(): Promise<Buffer[]>` — real providers like Replicate/Stability/OpenAI are often URL-based and asynchronous, not synchronous buffer returns):

```
startGeneration(originalImageBuffer): Promise<{ providerJobId: string, status: 'PENDING' }>
getGenerationStatus(providerJobId): Promise<GeneratedImageResult[]>

GeneratedImageResult = {
  status: 'READY' | 'PENDING' | 'FAILED',
  imageUrl?: string,           // provider-hosted result
  imageBuffer?: Buffer,        // or a raw buffer, if the provider returns one directly
  providerJobId?: string,
  metadata?: Record<string, unknown>, // model/version, prompt, seed, etc.
  error?: string,
}
```

The stub provider returns 4 `READY` results synchronously (still fits the interface, just resolves immediately); job orchestration, DB states (`ai_image_jobs` / `ai_generated_images`), and the admin approval UI are provider-independent and can be fully built against this interface now. `ai_generated_images` gains `provider_job_id` and `metadata jsonb` columns to carry this through.

11. Cart, Checkout, Payment, Order Lifecycles (corrected: PENDING order + reservation, webhook-only confirmation)

- **Cart:** guest cart keyed by a `cart_session` cookie; on login, `guest cart + user cart` merged (dedupe by `product_id`) — same pattern for wishlist. Cart/PDP always show the live current price (no reservation, no lock — browsing doesn't touch inventory).
- **Checkout:** single page, sectioned (Contact → Address → Delivery [single free-shipping option, no radio needed] → Payment → Summary); Buy Now creates a single-item cart and jumps straight in.
- **Order state machine (corrected):** `PENDING_PAYMENT` → `CONFIRMED` → `PROCESSING` → `SHIPPED` → `OUT_FOR_DELIVERY` → `DELIVERED` (+ `PAYMENT_FAILED`, `EXPIRED`, `CANCELLED`, `RETURN_REQUESTED`, `REFUNDED` side-branches), every transition appended to `order_status_history`, driving both "My Orders" tabs and the track-order timeline.

Checkout → Payment → Confirmation flow (corrected two-phase split):

1. Checkout submit → single transaction:
 - insert order (`status=PENDING_PAYMENT`), `order_items` snapshot current `gold_value/diamond_value/making_charge/gst/selling_price + gold_rate_id` per item
 - insert one `stock_reservations` row per item (`status=ACTIVE`, `expires_at = now + RESERVATION_TTL_MINUTES`)
 - coupon NOT yet marked used
 - `PaymentService.createIntent(order)` → frontend redirects/embeds per provider
- 2a. Payment gateway webhook fires → verify signature → look up payment by `provider_ref` (`UNIQUE` constraint makes a repeat delivery a safe no-op) → single transaction:
 - `order.status: PENDING_PAYMENT` → `CONFIRMED`
 - `stock_reservations: ACTIVE` → `CONFIRMED`, and `products.stock_quantity` decremented for
 - `coupon_usage` row inserted (coupon now actually consumed)
 - `payments` row written/updated with provider payload
 - enqueue confirmation email job (§12)
 - Never trust the frontend's payment-success redirect to do any of this — only the webhook
- 2b. Reservation-expiry sweep (scheduled pg-boss job, e.g. every minute):
 - finds `stock_reservations` where `status=ACTIVE` AND `expires_at < now`
 - releases them (`status=RELEASED`, no stock impact since nothing was decremented yet)
 - if the parent order is still `PENDING_PAYMENT`, moves it to `EXPIRED`

Available stock shown to shoppers is always `products.stock_quantity - SUM(ACTIVE reservations)`, so a second shopper can't reserve what's already held in someone else's pending checkout — this is what actually prevents overselling low-stock/one-of-a-kind pieces.

- **Delivery estimate on PDP/checkout:** since there's only one shipping method and no per-pincode serviceability table in the schema, the "Enter Pincode → Check" action does a lightweight format/length validation (and optionally a state/zone lookup from a small static table if COD-style exclusions are ever needed) and returns a configurable estimate (e.g. `SHIPPING_ESTIMATE_MIN_DAYS / MAX_DAYS` from

settings) rather than true carrier serviceability — flag if per-pincode serviceability is actually required later, since that would need its own table.

11a. Reviews Lifecycle

Customer views a DELIVERED order → "Write a Review" on an item in it → POST /products/:id/reviews {rating, title, body, order_item_id} → server verifies order_item belongs to user AND order_item.order_id's orders.status = 'DELIVERED' (the real fulfillment field — see §3) AND no existing review for that order_item → insert with status=PENDING, is_verified_purchase=true → admin reviews queue (/admin/reviews) → approve → status=APPROVED, products.rating_avg/rating_count updated in the same transaction → now visible on PDP (GET /products/:id/reviews returns APPROVED only, paginated)

Reviews submitted without a matching delivered order are rejected outright (no unverified reviews) — this keeps the moderation queue small and ratings trustworthy for a launch-stage store. Eligibility (`orders.status = DELIVERED`) is driven by the shipping/tracking updates described next.

11b. Shipping Architecture

Same abstraction pattern as `OTPPProvider` / `PaymentService` / `MetalRateProvider` / `AIImageProvider` — stub now, real courier (Shiprocket/Delhivery/etc.) later, nothing else in the codebase changes when it's swapped in.

ShippingProvider interface:

```
createShipment(order): Promise<{ providerShipmentId, trackingNumber?, courierName?, status }>
getTracking(providerShipmentId): Promise<{ status, events: [...], estimatedDelivery? }>
cancelShipment(providerShipmentId): Promise<{ status }>
```

Flow: order reaches `CONFIRMED` / `PROCESSING` → admin (or an automatic trigger, TBD when this phase is built) calls `ShippingProvider.createShipment()` → `shipments` row created (`provider`, `provider_shipment_id`, `tracking_number`, `courier_name`, `status`) → `order_status_history` appends `SHIPPED`. Inbound courier webhook (POST /api/v1/shipping/webhook , signature-verified and idempotent on the courier's own event id, same discipline as the payment webhook) updates the shipment's status/tracking events and mirrors relevant transitions into `order_status_history` (`OUT_FOR_DELIVERY`, `DELIVERED`) — reaching `DELIVERED` is what makes the order's items eligible for review submission (§11a).

12. Email Architecture

Every transactional event (order confirmed, shipped, delivered, etc.) enqueues an `email_jobs` row (via `pg-boss`) instead of sending inline — a worker sends via `EmailProvider` (stubbed to console/log in dev) and

writes `email_logs`. Failures retry with backoff and never delay the HTTP response that triggered them.

13. Performance & SEO Plan

- Route-based code splitting per page (`React.lazy` + Vite dynamic import) for both `web` and `admin`.
- Server sends only fields each screen needs (list endpoints return list-shaped DTOs, not full product rows).
- Fonts: self-hosted variable fonts (serif for headings, sans for body), subset, `font-display: swap`, only the two critical weights preloaded.
- Images: AVIF/WebP with `srcset`, lazy below the fold, correctly sized per breakpoint, never original files in listings.
- Compression (Brotli/gzip) at the reverse proxy; cache headers tuned per asset type (immutable hashed JS/CSS, short-TTL API responses where cacheable).
- SEO: slugged URLs (`/rings/celeste-diamond-ring`), request-time SSR for Home/PLP/Collections/PDP with `<title> /meta/JSON-LD` (Product, BreadcrumbList, Organization) rendered server-side into initial HTML (§1a — not client-injected after the fact), `sitemap.xml` generated from published products/categories (including the hierarchical category tree), `robots.txt`.

14. Phased Build Plan (one phase discussed & built at a time)

Cross-phase acceptance criteria (correction: these apply to every relevant phase as it's built, not deferred to the final phase):

- **[PERF]** — any phase that ships or changes a public page: run Lighthouse against it before the phase is considered done; no regression below the target thresholds (Performance ≥ 90 , CLS near-zero). Not deferred to Phase 20.
- **[SEO]** — any phase that ships a new public route: correct canonical URL, title/meta description, and required JSON-LD must exist before the phase is done.
- **[SEC]** — any phase that ships a new endpoint: input validation, correct auth/role check, and rate limiting (where relevant) are reviewed before the phase is done; any new webhook must have signature verification + idempotency demonstrated with a test before the phase closes.

Phase	Scope	Criteria
0	Repo scaffolding: monorepo folders, Vite apps, Express skeleton, env config, lint/format tooling	
1	Database schema + migrations (all tables from §3, incl. <code>stock_reservations</code> , <code>page_cache</code> , hierarchical <code>categories</code> , <code>addresses</code> partial unique index) + seed script	
2	Backend core: db pool, error handling middleware, admin auth (email/password + session + roles)	[SEC]
3	Customer OTP auth: <code>otp_verifications</code> , <code>OTPPProvider</code> (stub + MSG91), send/verify endpoints, session creation, guest-cart-session cookie	[SEC]
4	Product & catalog backend: products/categories (hierarchical)/collections CRUD (admin) + public listing/detail/search endpoints with pagination & indexes; <code>stock_reservations</code> table + available-stock calc	[PERF] [SEC]
5	Live Pricing Engine: <code>MetalRateProvider</code> (stub) + scheduled gold-rate sync job, <code>PricingService</code> (gold/diamond value calc, purity multiplier), <code>gold_rates</code> / <code>diamond_rates</code> / <code>product_price_history</code> tables, admin pricing settings + live preview endpoint, price-lock toggle	
6	Storefront shell (CSR): header (search, wishlist, account, cart), homepage skeleton wired to <code>/api/v1/homepage</code> , design tokens/typography	[PERF]
7	PLP + PDP (web, CSR): filters sidebar, sort, product cards, PDP layout, price breakup (reads computed <code>gold_value</code> / <code>diamond_value</code> / <code>selling_price</code>)	[PERF]
8	SSR Rendering Layer (§1a): <code>entry-server</code> / <code>entry-client</code> , <code>web/server</code> render process, TanStack Query dehydration/hydration for Home/PLP/Collections/PDP, <code>page_cache</code> table + invalidation hooks wired into product/category/price mutations, server-rendered meta + JSON-LD, <code>sitemap.xml</code> / <code>robots.txt</code>	[PERF] [SEO]
9	Cart + Wishlist (guest + merge-on-login)	[PERF]
10	Checkout flow + address management (partial-unique default-address constraint) + Buy Now + inventory reservation creation (<code>PENDING_PAYMENT</code> order + <code>stock_reservations</code> row)	[SEC]
11	Payment: <code>PaymentService</code> abstraction + stub provider + idempotent signature-verified webhook (confirms order, converts reservation → stock decrement, finalizes coupon usage) + reservation-expiry sweep job	[SEC]
12	Orders: My Orders, track order timeline, invoice	
13	Shipping: <code>ShippingProvider</code> abstraction (stub), shipment creation on order confirm, tracking, cancellation, idempotent signature-verified courier webhook	[SEC]
14	Admin panel: products (incl. jewellery attributes, SEO fields, live pricing preview from Phase 5), categories (tree UI), collections, orders, customers	[SEC]

Phase	Scope	Criteria
15	AI image job architecture: job queue, worker, restructured <code>AIImageProvider</code> (structured results, async-capable), admin approval/regenerate/reorder UI	
16	Homepage admin (dynamic sections, reordering, enable/disable) — triggers <code>page_cache</code> invalidation for Home	[SEO]
17	Reviews: submission gated on <code>orders.status = DELIVERED</code> , admin moderation queue, PDP display, denormalized rating aggregate	[SEC]
18	Email notification architecture (queue + templates + stub provider)	
19	Coupons, audit logs, admin users/roles	[SEC]
20	Full hardening pass: Lighthouse across all page types, sitemap/robots final check, full security review — a final audit on top of criteria already met per-phase, not the first time these are addressed	[PERF] [SEO] [SEC]

Each phase will be scoped into its own discussion before implementation, per the user's instruction not to build everything in one pass.

Verification Approach (ongoing, per phase)

- Backend: integration tests hitting real Postgres (test DB) for each service/repository; `npm run lint + tsc --noEmit` if TypeScript is adopted.
- Frontend: manual verification via the `run` skill (start dev servers, exercise the golden path in-browser) plus Lighthouse checks once pages exist, since this spec explicitly grades success on real Lighthouse scores, not just tests passing.
- Webhooks (payment, shipping): tested by replaying the same signed payload twice and asserting the second delivery is a safe no-op (idempotency), plus a bad-signature payload asserting rejection.
- Inventory reservations: tested by simulating two concurrent checkouts against the last unit of a low-stock product and asserting only one can reserve it; a manually-expired reservation is asserted to release stock back to availability.
- SSR (Phase 8 onward for the four public route types): verified by curling the route directly (no JS execution) and asserting the response HTML already contains the product/category name, price, and JSON-LD — not just an empty root div.
- No phase is considered done until its slice is demonstrably working end-to-end (API → DB, or UI → API) **and** its [PERF]/[SEO]/[SEC] tags from §14 are satisfied, not just written.